

Systèmes microprogrammés

1re STI2D · Cours · Arduino et ESP32 · Architecture, IDE, langages et mémoires

Un système microprogrammé décide par un programme, pas par un câblage figé. Ce cours décrit l'architecture, les cartes Arduino Uno et ESP32, l'environnement de développement, les langages et les mémoires. Le programme lui-même s'écrit en C++ : c'est l'objet de l'autre cours.

1. CÂBLÉ OU PROGRAMMÉ

Système câblé. Le comportement est fixé par les liaisons et les composants. Pour le modifier, il faut recâbler.

Système microprogrammé. Le comportement est un programme stocké en mémoire. On le change en téléversant un nouveau programme, sur le même matériel.

Le programme est une suite d'instructions. Le processeur les exécute l'une après l'autre.

CHAÎNE D'INFORMATION

```
capteur → entrée → programme
                ↓
                sortie → actionneur
```

Capteurs : bouton, lumière, température, distance...

Actionneurs : LED, moteur, buzzer, afficheur...

Le programme lit, décide, puis commande. C'est le même schéma pour tout système embarqué.

2. LE MICROCONTRÔLEUR

Un **microcontrôleur** réunit sur une seule puce le processeur, les mémoires et les périphériques. Un microprocesseur n'est que l'unité de calcul : les mémoires et les entrées-sorties sont alors des circuits à part.

- Processeur : lit et exécute les instructions
- Horloge : cadence l'exécution, en mégahertz
- Mémoires : programme et données
- Périphériques : entrées-sorties, convertisseur, minuteurs, liaison série

LES BROCHES

Une **broche GPIO** (entrée-sortie à usage général) est un fil dont le programme choisit le rôle : entrée ou sortie.

- Sortie : le programme impose un niveau haut ou bas
 - Entrée numérique : le programme lit un niveau haut ou bas
 - Entrée analogique : le programme lit une tension, convertie en nombre
- Le niveau haut vaut 5 V sur un Arduino Uno, et 3,3 V sur un ESP32. Le niveau bas vaut 0 V.

3. DEUX CARTES, LE MÊME PRINCIPE

Arduino Uno et ESP32 se programment de la même manière. Ce qui change, ce sont la tension, la taille des mémoires, la finesse de la mesure et les communications. Le texte du programme garde la même forme.

	Arduino Uno	ESP32
Microcontrôleur	ATmega328P	ESP32, deux cœurs
Horloge	16 MHz	jusqu'à 240 MHz
Tension des broches	5 V	3,3 V
Flash (programme)	32 Ko	souvent 4 Mo
RAM	2 Ko	environ 520 Ko
Données conservées	EEPROM, 1 Ko	mémoire NVS
Mesure analogique	0 à 1023 (10 bits)	0 à 4095 (12 bits)
Entrées analogiques	A0 à A5	certaines broches GPIO
Réseau	non	Wi-Fi et Bluetooth
LED souvent prévue	broche 13	aucune broche imposée

Ne jamais relier une tension de 5 V à une broche d'ESP32. Sur l'ESP32, plusieurs broches (souvent les numéros 34 à 39) sont des entrées seules : on peut y lire une valeur, pas y commander une sortie. La broche 13 d'un Arduino Uno pilote en général une LED déjà soudée sur la carte : c'est la broche des premiers essais.

4. L'IDE ARDUINO

L'**IDE** (environnement de développement intégré) réunit les outils qui font passer du texte au programme dans la carte.

- Éditeur : on écrit le programme, un croquis de fichier .ino
- Vérifier : le compilateur traduit le C++ et affiche les erreurs
- Téléverser : le résultat est écrit dans la flash, la carte redémarre
- Moniteur série : affiche les textes envoyés par la carte

Le dossier du croquis doit porter le même nom que le fichier .ino.

Dans le menu Outils, on choisit le type de carte et le port qui apparaît quand le câble USB de données est branché.

5. LES LANGAGES

On décrit d'abord le comportement en français, étape par étape : c'est l'algorithme. Ensuite on le traduit.

Le cours écrit en C++. Les noms setup, loop, pinMode ou digitalWrite viennent des bibliothèques Arduino. Ce n'est pas un autre langage.

- Le C++ est compilé : la carte exécute des instructions machine. L'exécution est rapide.

- MicroPython, possible sur un ESP32, est interprété ligne par ligne. Ce n'est pas le langage de ce cours.

```
C++ → compilateur → langage machine
                ↓
                écrit dans la flash
```

6. LES MÉMOIRES

Une mémoire volatile perd son contenu hors tension. Une mémoire non volatile le conserve. Trois rôles, trois mémoires.

Mémoire	Hors tension	Qui y écrit	Ce qu'on y trouve
Flash	conservée	le téléversement	le programme
RAM	perdue	le programme, en marche	les variables
EEPROM (Uno)	conservée	le programme	un réglage à garder
NVS (ESP32)	conservée	le programme	même usage que l'EEPROM

Dans d'anciens documents, la mémoire du programme s'appelle ROM. Sur ces cartes, c'est de la flash : elle se réécrit à chaque téléversement. La RAM de l'Arduino Uno est petite : 2 Ko. Un int y occupe 2 octets, et 4 octets sur un ESP32.

OÙ VA CHAQUE INFORMATION

```
const int BROCHE = 13; // valeur fixe
int mesure = 0;        // variable, RAM

void setup() { }        // en flash, une fois
void loop() {           // en flash, sans fin
  mesure = analogRead(A0);
}
```

À la coupure, le programme reste. La variable mesure disparaît.

7. ENTRÉES ET SORTIES

Instruction	Effet
pinMode(p, OUTPUT)	la broche devient une sortie
digitalWrite(p, HIGH)	niveau haut : 5 V ou 3,3 V
digitalWrite(p, LOW)	niveau bas : 0 V
digitalRead(p)	lit un niveau haut ou bas
analogRead(p)	lit une tension, renvoie un entier
delay(500)	attend 500 ms, tout est figé

analogRead ne renvoie pas des volts, mais un code. Le PWM (analogWrite sur Uno) hache le niveau haut pour doser une LED ou un moteur.

8. LA MESURE ANALOGIQUE

Le convertisseur découpe la plage de tension en 2ⁿ échelons. Plus n est grand, plus l'échelon est fin.

	Uno	ESP32
Bits n	10	12
Codes	0 à 1023	0 à 4095
Pleine échelle	5 V	3,3 V

On estime la tension en rapportant le code à la pleine échelle.

9. DU BRANCHEMENT À L'EXÉCUTION

Câble de données, carte choisie, port choisi

↓
Vérifier : traduction en langage machine

↓
Téléverser : écriture dans la flash

↓
setup() une fois, puis loop() sans fin

À la mise sous tension, un petit programme déjà en flash (le bootloader) démarre, puis lance le programme téléversé.

setup() prépare la carte. loop() lit, décide, commande, et recommence.

10. LE MONITEUR SÉRIE

La carte et l'ordinateur peuvent s'échanger du texte par le câble USB. Serial.begin(9600) ouvre la liaison à 9600 bauds. Le moniteur série doit être réglé sur le même nombre, sinon les caractères sont illisibles. 9600 et 115200 sont deux vitesses courantes. On affiche une valeur avec Serial.println.

11. UN PREMIER PROGRAMME

```
const int BROCHE_LED = 13;
void setup() {
  pinMode(BROCHE_LED, OUTPUT);
}
void loop() {
  digitalWrite(BROCHE_LED, HIGH);
  delay(500);
  digitalWrite(BROCHE_LED, LOW);
  delay(500);
}
```

À RETENIR

1. Microprogrammé : on change le comportement en changeant le programme.
2. Le programme est en flash. Les variables sont en RAM et disparaissent hors tension.
3. Uno : 5 V et codes de 0 à 1023. ESP32 : 3,3 V et codes de 0 à 4095.
4. L'IDE compile, téléverse, puis le moniteur série sert à observer.

PIÈGES FRÉQUENTS

- Mettre du 5 V sur une broche d'ESP32
- Câble USB qui ne transporte pas les données
- Mauvaise carte ou mauvais port
- Vitesse du moniteur différente de Serial.begin
- Dossier et fichier .ino de noms différents
- Commander une sortie sur une broche d'entrée seule
- Croire que la RAM survit à la coupure